

Part 1

1.

My proposed row key is hash(VIN) + “#” + ModelYear + “#” + Make + “#” + City. If the row key begins with an attribute like model year, new data will constantly be written to the same tablet, which will cause hotspotting. This is when one node gets overloaded. By hashing the VIN as the prefix of the row key, writes are distributed evenly across tablets, no node gets too much write traffic, and parallel writes work fine. The VIN is ideal because it is unique for each car.

2. Column families should group attributes that are accessed together. Bigtable stores column families separately which means this would improve the efficiency.

Here are some column families:

Vehicle Info: VIN, Make, Model, Model Year, Electric Vehicle Type, Electric Range
Registration Info: County, City, State, Postal Code, Legislative District, DOL Vehicle ID
Utility Info: Electric Utility
Metadata: 2020 Census Tract

This aids in distributed read performance because it reduces disk input/output, improves compression efficiency, faster scans for analytical queries, and more efficient caching.

3. The pros of using a hashed shard key based on VIN is even data distribution, high write throughput, and evenly distributed cluster utilization. The cons of this is rows with similar attributes are scattered across nodes and the scans become more complex

The pros of using a range-based shard key based on model year are more efficient range scans and it is better for analytical queries. The cons of this are hotspotting, uneven cluster utilization, and reduced write throughput.

Part 4

The average latency for /insert-fast is 0.007880558967590332 and the average latency for /insert-safe is 0.023726811408996584.

According to my latency numbers and the CAP theorem, the w=1 write preference prioritizes high availability and performance over consistency and durability. A perfect scenario for using w=1 is when a bunch of electric vehicles send their GPS coordinates every second. In this case, there is a lot of data and even if we lose a data point, it is not a big deal. The durability of the data is not as important.